

Arquitectura de Referencia — Open Banking sobre Red Hat OpenShift

Versión: 1.0.0

Fecha: 2026-06-25

Plataforma objetivo: Red Hat OpenShift Container Platform 4.14+

Autor: Juan Pablo Garzon

Revisor: Ronald Lopez

Estado: Documento de referencia técnica

Tabla de Contenidos

1. [Visión General](#)
 2. [Implementaciones Red Hat para Open Banking](#)
 3. [Componentes de la Arquitectura](#)
 4. [Diagrama de Arquitectura Lógica](#)
 5. [Diagrama de Flujo de Petición \(Request Flow\)](#)
 6. [Diagrama de Componentes — Capa de API Management](#)
 7. [Diagrama de Componentes — Capa de Identidad y Consentimiento](#)
 8. [Arquitectura de Despliegue en OCP](#)
 9. [Modelo de Datos y Bases de Datos](#)
 10. [Políticas de API \(3scale\)](#)
 11. [Consideraciones de Seguridad y FAPI](#)
 12. [Escalabilidad y Alta Disponibilidad](#)
 13. [Manifiestos OCP de Referencia](#)
-

1. Visión General

Esta arquitectura de referencia describe el despliegue de Open Banking / Open Finance usando exclusivamente el ecosistema **Red Hat** sobre **OpenShift Container Platform (OCP)**. Los servicios de negocio financiero (Gestión de Consentimientos, Notificaciones de Eventos, APIs de Cuentas, Pagos y VRP) son implementados como microservicios **Quarkus 3.8** nativos para la nube.

La propuesta tiene tres objetivos estratégicos:

- **Vendor único** — toda la infraestructura, desde el gateway hasta el runtime de los microservicios, usa productos con soporte Red Hat. Esto simplifica los acuerdos de licenciamiento, el soporte técnico y las actualizaciones de seguridad.
- **Cumplimiento regulatorio por diseño** — Keycloak incluye perfiles FAPI certificados por la OpenID Foundation. Las políticas de seguridad no son configuraciones manuales sino artefactos versionados en Git y validados en tiempo de despliegue.
- **Arquitectura cloud-native** — todos los componentes operan como contenedores en OCP, escalan horizontalmente de forma autónoma (HPA), se despliegan mediante GitOps (ArgoCD) y se monitorean con el stack de observabilidad nativo de OpenShift.

Capacidades principales

Capacidad	Componente Red Hat	Estándar cubierto
Dynamic Client Registration (DCR)	Keycloak 26 (nativo)	RFC 7591, FAPI 1.0
Consent Management	ob-consent-service (Quarkus)	Open Banking UK, PSD2
Event Notifications	ob-event-notifications (Quarkus)	RFC 8417 (SET), SSE
mTLS Enforcement	Red Hat Service Mesh (Istio)	FAPI 1.0 Advanced
API Gateway	Red Hat 3scale API Management	OAuth2, PKCE, FAPI
SCA (Strong Customer Authentication)	Keycloak Auth Flows + WebAuthn	PSD2 RTS, FAPI 2
Mock Bank APIs	ob-bank-backend (Quarkus)	Open Banking UK v4.0
Self-Care Portal	React SPA + nginx	—

Versiones de componentes

Producto	Versión	Canal de soporte
Red Hat OpenShift Container Platform	4.14 (EUS)	Red Hat
Red Hat 3scale API Management	2.14	Red Hat (Operador)
Keycloak (Red Hat SSO 2)	26.x	Red Hat
Red Hat Service Mesh	2.6 (Istio 1.20)	Red Hat (Operador)
Red Hat AMQ Streams	2.7 (Kafka 3.7)	Red Hat (Operador)
Quarkus	3.8.x (LTS)	Red Hat Build of Quarkus
MySQL	8.0 (via MySQL Operator)	Community / Percona
Flyway	10.x	Community

2. Implementaciones Red Hat para Open Banking

Antes de diseñar una arquitectura propia, es fundamental conocer qué ofrece Red Hat de forma nativa para el sector financiero. Esta sección documenta las capacidades ya disponibles en los productos Red Hat que son directamente aplicables a Open Banking, evitando desarrollos innecesarios y apoyándose en certificaciones y validaciones del ecosistema.

2.1 Keycloak y FAPI (Financial-grade API)

Red Hat distribuye Keycloak (desde v21) con **perfiles globales de seguridad pre-configurados** para Open Banking. Estos perfiles cumplen los estándares internacionales requeridos para banca abierta:

Perfil Global	Estándar	Capacidades incluidas
fapi-1-baseline	FAPI 1.0 Baseline	PKCE obligatorio, mTLS, redirect URI exactas

Perfil Global	Estándar	Capacidades incluidas
fapi-1-advanced	FAPI 1.0 Advanced	Request Objects firmados (JAR), PAR, mTLS bound tokens
fapi-ciba	FAPI CIBA	Backchannel authentication, binding_message
fapi-2-security-profile	FAPI 2 Security Profile	DPoP, PAR obligatorio, PKCE S256
oauth-2-1	OAuth 2.1	PKCE obligatorio, sin Implicit Flow

Activación en Keycloak:

```
# Realm client policy para FAPI 1.0 Advanced
{
  "name": "openbanking-fapi-policy",
  "conditions": [
    { "condition": "client-roles", "configuration": { "roles": ["openbanking-tpp"] } }
  ],
  "profiles": ["fapi-1-advanced"]
}
```

2.2 Keycloak y PSD2 / Open Finance Colombia

Para el contexto regulatorio colombiano (Superintendencia Financiera de Colombia — SFC) y la adopción del marco PSD2 europeo como referencia, Keycloak provee nativamente los siguientes mecanismos sin necesidad de desarrollo adicional:

- **DCR (Dynamic Client Registration)** — RFC 7591, RFC 7592
- **PAR (Pushed Authorization Requests)** — RFC 9126
- **DPoP** — RFC 9449 (feature `dpop` en Keycloak 24+)
- **Autenticación paso a paso (Step-Up)** — mapeado a niveles de autenticación SCA
- **WebAuthn / Passkeys** — FIDO2, W3C WebAuthn Level 3
- **CIBA (Client Initiated Backchannel Authentication)** — flujos de pago asincrónico

2.3 Red Hat 3scale y Open Banking

3scale actúa como el punto de control centralizado para todas las APIs Open Banking expuestas hacia TPPs. Su rol no es solo proxy HTTP — es la capa de gobierno que aplica políticas de negocio (planes, límites, monetización) sobre las APIs técnicas expuestas por los microservicios Quarkus. 3scale API Management v2.14 incluye:

- **Políticas nativas** para rate limiting por TPP, transformación de headers, validación JWT
- **Developer Portal** para onboarding self-service de TPPs con flujo de aprobación
- **Analytics** por API, por plan, por aplicación — requerido para reporte regulatorio
- **ActiveDocs (OpenAPI)** publicación del catálogo de APIs Open Banking

2.4 Red Hat en el ecosistema financiero global

Red Hat no es nuevo en el sector financiero. Las siguientes referencias validan la madurez de la plataforma para casos de uso bancarios en producción, incluyendo proyectos de modernización similares al objetivo de este documento:

Cliente / Partner	Caso de uso	Tecnología RH
Emirates NBD	Plataforma cloud para servicios bancarios innovadores	OCP, JBoss EAP
Banco Galicia	Plataforma de IA para NLP en banca digital	OCP, Red Hat AI
Finastra	Open Finance Platform (120+ bancos globales)	OCP, 3scale
Temenos	Core bancario en contenedores	OCP, JBoss EAP
ACI Worldwide	Pagos en tiempo real (ISO 20022)	OCP, AMQ Streams
Capgemini Blueprint	Modernización bancaria (85% más rápida)	OCP, Ansible

3. Componentes de la Arquitectura

La solución se divide en cinco capas lógicas, cada una con responsabilidades claras y aisladas. Esta separación permite escalar, actualizar y monitorear cada capa de forma independiente sin afectar a las demás.

3.1 Capa de API Management — Red Hat 3scale

Esta capa es el único punto de entrada para los TPPs externos. Ningún TPP accede directamente a los microservicios Quarkus — toda llamada pasa obligatoriamente por APIcast, donde se valida el token OAuth2, se verifica el consentimiento, se aplica el rate limiting y se enriquecen los headers antes de reenviar la petición al backend.

```
financial-services-accelerator-local/open-banking-rh/
├── (integración con 3scale via admin REST API)
├── Políticas de mediación via Camel K

Componentes 3scale:
├── APIManager (operador)      # Gestión central de APIs
├── Developer Portal           # Self-service onboarding TPPs
├── API Gateway (APIcast)      # Proxy Nginx con políticas OIDC
├── Backend (Redis + MySQL)    # Almacenamiento estado 3scale
├── Zync                       # Sincronización Keycloak ↔ 3scale
```

3.2 Capa de Identidad y Consentimiento — Keycloak + Quarkus

Esta es la capa más crítica de la arquitectura desde el punto de vista regulatorio. Keycloak gestiona las identidades de clientes y TPPs, emite y valida los tokens OAuth2/OIDC con perfiles FAPI. Los microservicios Quarkus implementan el ciclo de vida completo de los consentimientos (creación, autorización, validación, revocación) y el sistema de notificaciones de eventos (SET tokens RFC 8417) que informan a los TPPs sobre cambios de estado.

```

open-banking-rh/
├── ob-consent-service/           # Gestión ciclo de vida consentimientos
│   ├── ConsentResource.java     # CRUD consentimientos (JAX-RS)
│   ├── ConsentAdminResource     # Admin: búsqueda y revocación
│   ├── ConsentService.java      # Lógica de negocio (CDI)
│   ├── ConsentEntity.java       # Hibernate Panache (MySQL)
│   └── ConsentEventPublisher    # Kafka → AMQ Streams
├── ob-event-notifications/      # Notificaciones SET (RFC 8417)
│   ├── EventSubscriptionResource # Suscripciones webhook TPP
│   ├── ConsentEventConsumer     # Kafka consumer
│   └── EventNotificationEntity   # Persistencia (MySQL)
└── Keycloak Realm "openbanking"
    ├── Authentication Flow SCA   # Password + WebAuthn/OTP
    ├── Client Policy FAPI 2     # Perfiles seguridad globales
    ├── DCR Endpoint             # Registro dinámico TPPs
    └── JWKS Endpoint            # Claves públicas para verificación

```

3.3 Capa de APIs de Negocio — Quarkus Backend

Implementa los contratos de la especificación Open Banking UK v4.0 como APIs REST. En la arquitectura actual utiliza datos de prueba (mock), pero el diseño permite sustituir las implementaciones por llamadas reales al core bancario sin cambiar los contratos de API expuestos hacia los TPPs. El patrón Strangler Fig permite migrar endpoint por endpoint en producción.

```

open-banking-rh/ob-bank-backend/
├── AccountResource.java         # GET /accounts, /accounts/{id}
│                               # GET /accounts/{id}/balances
│                               # GET /accounts/{id}/transactions
├── PaymentResource.java        # POST /payments/domestic-payments
│                               # GET /payments/domestic-payments/{id}
├── FundsConfirmationResource    # POST /funds-confirmations
└── VrpResource.java            # POST/GET /domestic-vrps

```

3.4 Capa de Mensajería — AMQ Streams (Kafka)

Kafka desacopla los microservicios y garantiza la entrega de eventos con durabilidad configurable. Los eventos de consentimiento fluyen desde **ob-consent-service** hacia **ob-event-notifications** a través del topic **ob-consent-events**, permitiendo que ambos servicios escalen de forma independiente. El topic **ob-audit-events** actúa como un log inmutable de auditoría regulatoria con retención configurable para cumplir los 5 años exigidos por la SFC.

```

Topics Kafka:
├── ob-consent-events           # Ciclo de vida consentimientos
│   └── payload: {eventType, consentId, clientId, userId, status}
└── ob-payment-events          # Estados de pagos

```

```

├── payload: {paymentId, status, accountId, amount}
├── ob-audit-events # Trazabilidad para cumplimiento SFC
├── payload: {action, actor, resource, timestamp, ipAddress}

```

3.5 Capa de Observabilidad

La observabilidad en Open Banking no es opcional — es un requisito regulatorio. Los bancos deben demostrar ante la SFC que pueden auditar cada acceso a datos de clientes, detectar anomalías en tiempo real y responder ante incidentes de seguridad. La arquitectura implementa los **tres pilares de observabilidad**: métricas, logs y trazas distribuidas, más un cuarto pilar específico de Open Banking: el **log de auditoría regulatoria**.

- **Prometheus + Grafana** — Métricas operacionales (latencia, throughput, errores HTTP). Dashboards de SLA regulatorio que muestran el cumplimiento del límite de 2 segundos por API.
- **Loki + Fluentd** — Agregación de logs de todos los pods. Loki (solución nativa de Grafana Labs, compatible con OCP) permite consultas de logs con el mismo lenguaje que Prometheus (LogQL), facilitando correlación directa entre métricas y logs en el mismo dashboard de Grafana.
- **Jaeger (Distributed Tracing)** — Trazas end-to-end desde el gateway hasta la base de datos, usando el `x-b3-traceid` como correlationId. Esencial para diagnosticar latencias y auditar flujos de autorización FAPI.
- **EFK Stack** — Elasticsearch + Fluentd + Kibana para el almacenamiento a largo plazo (5 años) de logs de auditoría regulatoria. Indexado por consentId, tppld y userId para búsquedas forenses.

Monitoring Stack:

```

├── Prometheus (OCP Monitoring Stack)
│   └── Metricas: latencia APIs, JVM heap, DB pools, Kafka consumer lag
├── Grafana
│   └── Dashboards: SLA regulatorio (p95/p99), TPP usage, consent funnel
├── Loki + Fluentd
│   └── Logs centralizados con LogQL – correlacion metricas/logs en Grafana
├── Jaeger (via Red Hat Service Mesh)
│   └── Trazas distribuidas – correlationId FAPI por peticion
├── EFK Stack (Elasticsearch + Fluentd + Kibana)
│   └── Audit trail regulatorio – retencion 5 anos – indexado por consent/tppl/user

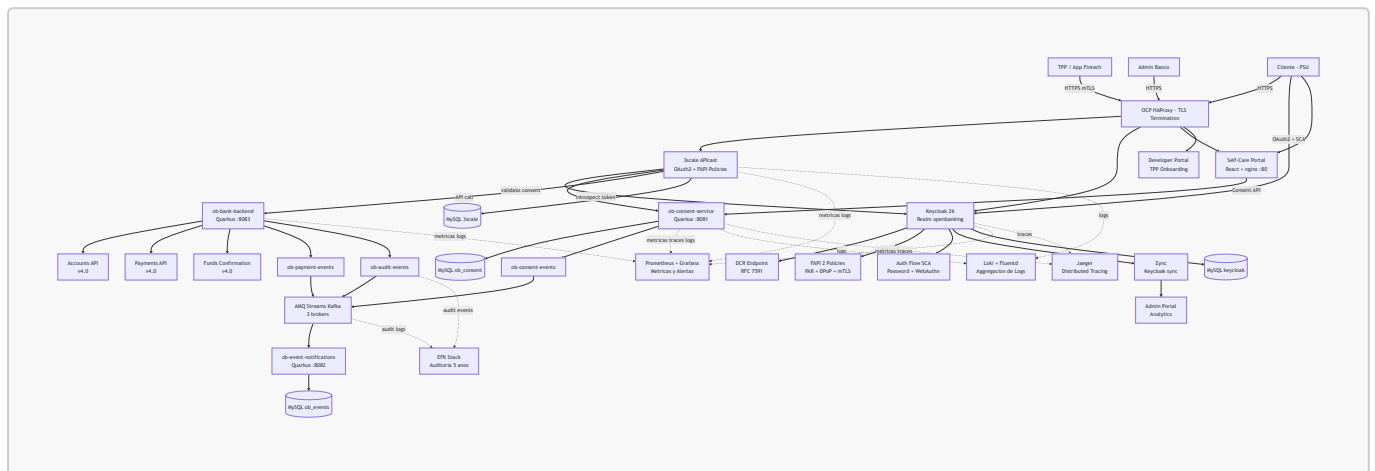
```

4. Diagrama de Arquitectura Lógica

El siguiente diagrama representa la vista lógica completa de la solución Open Banking sobre Red Hat OCP. Se organiza en **siete capas horizontales** que fluyen de arriba hacia abajo:

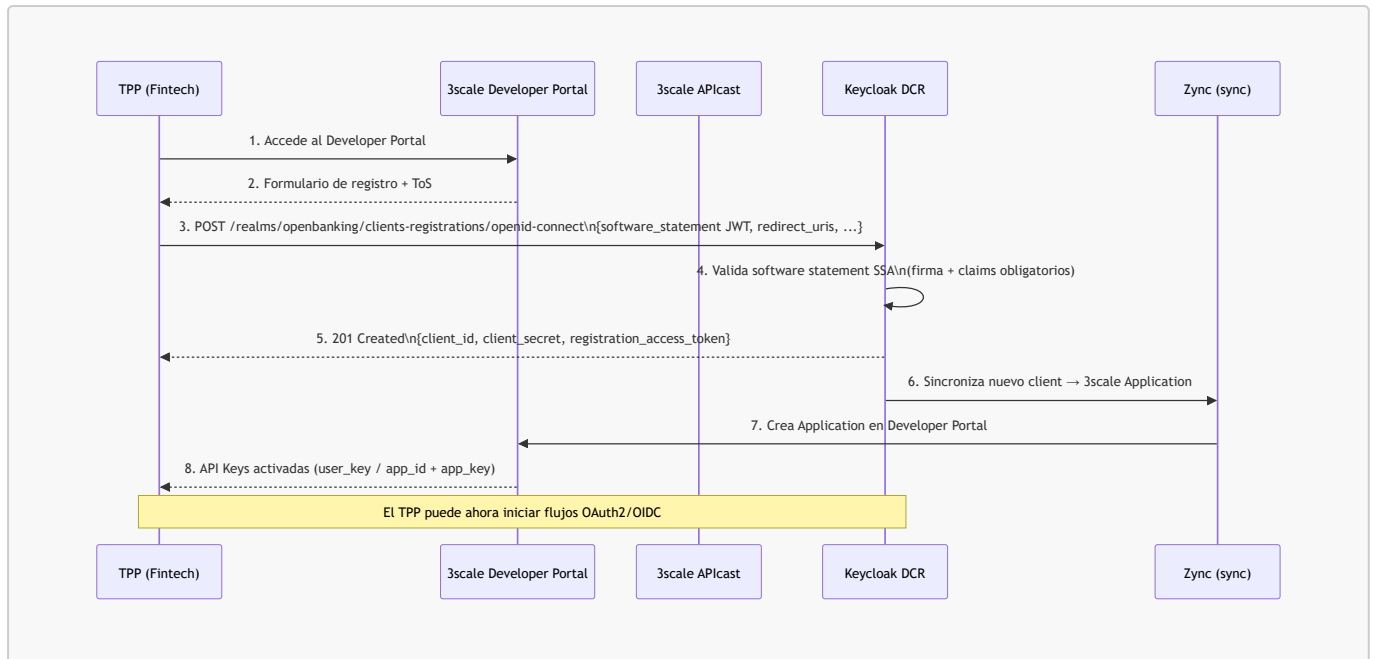
1. **Actores externos** — TPPs (terceros fintech), clientes bancarios (PSU) y administradores del banco.
2. **DMZ / Ingress** — Punto de entrada único con terminación TLS gestionado por el HAProxy de OCP.
3. **API Management (3scale)** — Capa de gobierno de APIs: autenticación OAuth2, rate limiting, políticas FAPI y Developer Portal para onboarding de TPPs.
4. **Identidad y Consentimiento** — Keycloak gestiona identidades, DCR, SCA y emite tokens. Los microservicios Quarkus administran el ciclo de vida de consentimientos y notificaciones de eventos.

- Las líneas continuas representan llamadas síncronas HTTP/gRPC; las líneas punteadas representan flujos de telemetría asíncrona.



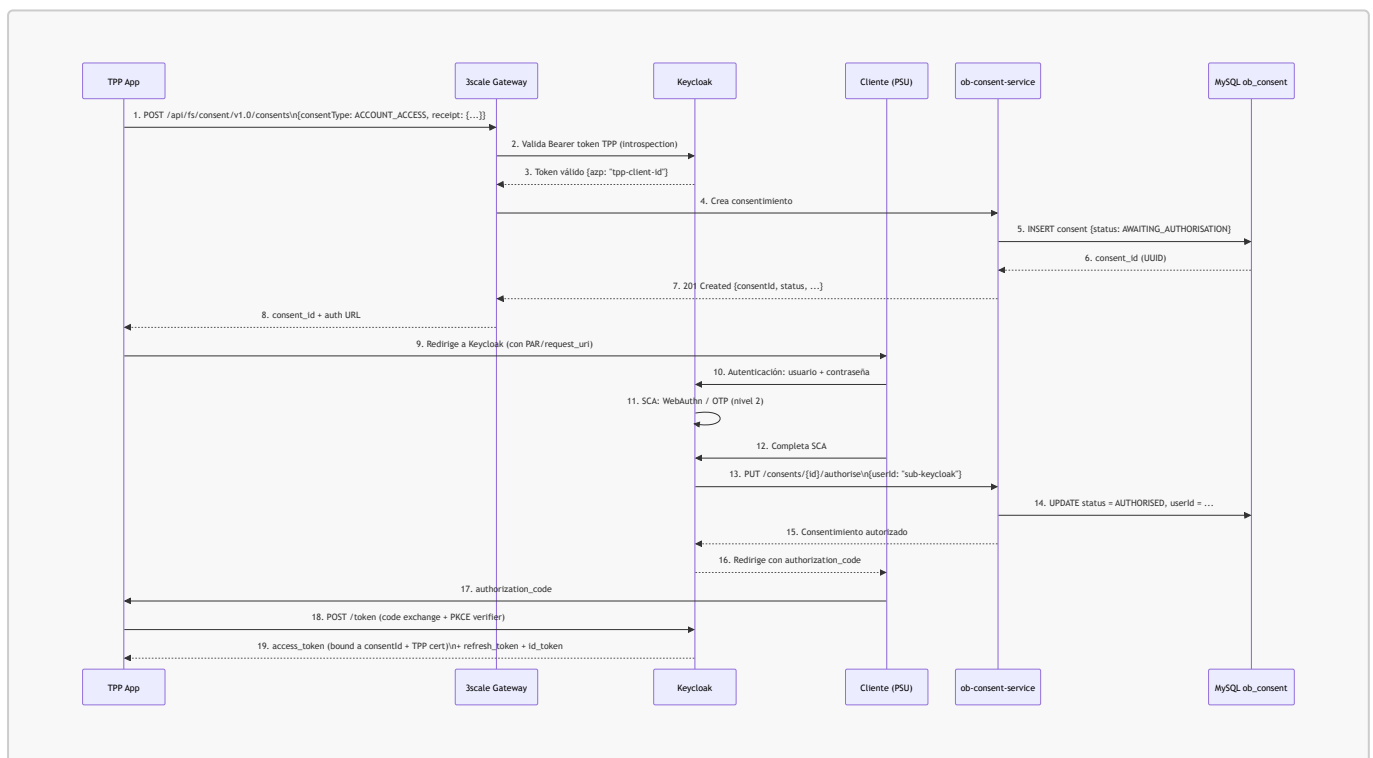
Esta sección documenta los cuatro flujos de petición más importantes de la arquitectura Open Banking. Cada diagrama de secuencia describe el camino exacto que recorre una petición, incluyendo los actores internos y los pasos de validación de seguridad en cada salto. Estos flujos son la base para definir las pruebas de integración end-to-end y para validar el cumplimiento regulatorio.

El registro dinámico de clientes es el proceso mediante el cual un TPP (fintech autorizado) obtiene sus credenciales OAuth2 para interactuar con las APIs del banco. En lugar de un proceso manual de configuración, DCR automatiza el registro mediante un software statement assertion (SSA) firmado por la entidad regulatoria. Keycloak valida esta firma, crea el client en el realm **openbanking** y sincroniza la aplicación en 3scale.



5.2 Flujo de Consentimiento del Cliente (OAuth2 + SCA)

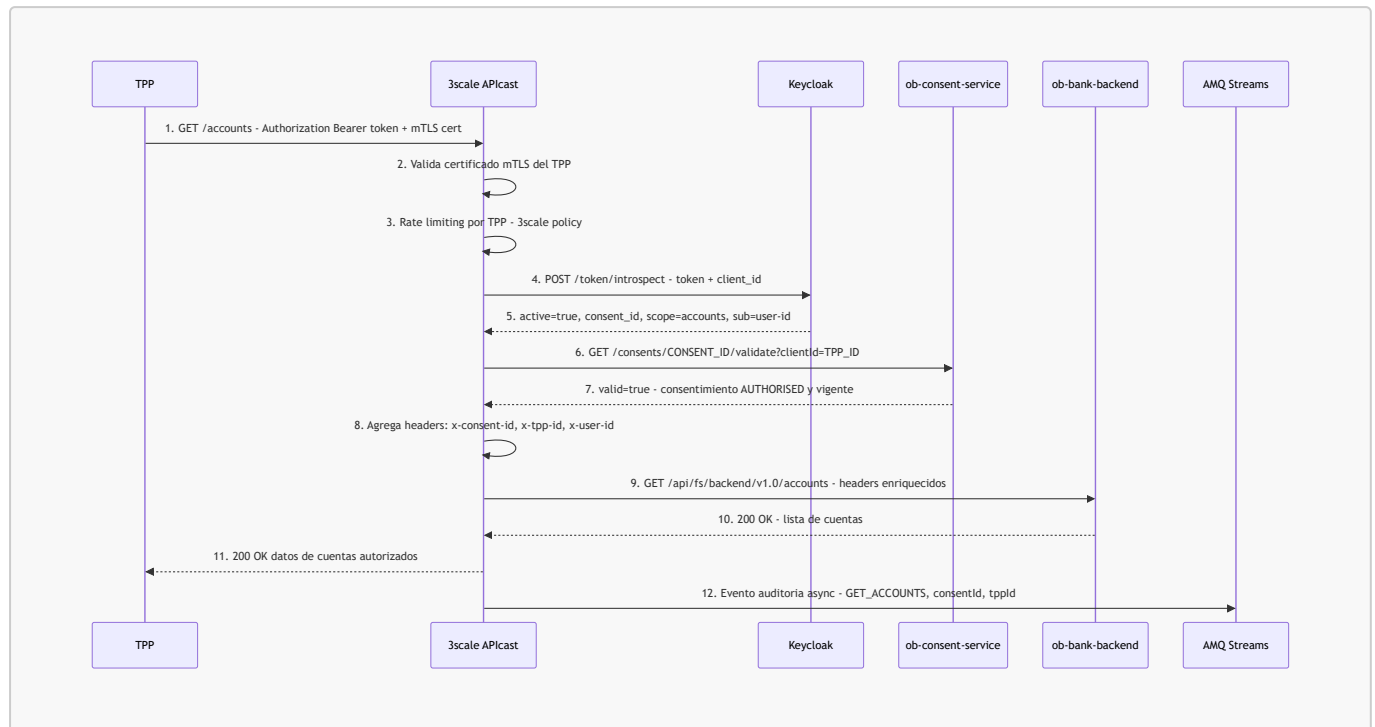
El consentimiento es el núcleo regulatorio de Open Banking. El cliente bancario (PSU) debe autorizar explícitamente qué datos puede acceder un TPP, por cuánto tiempo y con qué granularidad. Este flujo implementa el **Authorization Code Flow + PKCE** de OAuth2 con un paso de **SCA (Strong Customer Authentication)** — dos factores de autenticación requeridos por PSD2 RTS y el marco colombiano de la SFC. El consentimiento se persiste en MySQL y su estado se sincroniza a través de Kafka hacia el sistema de notificaciones.



5.3 Flujo de Llamada a API Protegida (Cuentas)

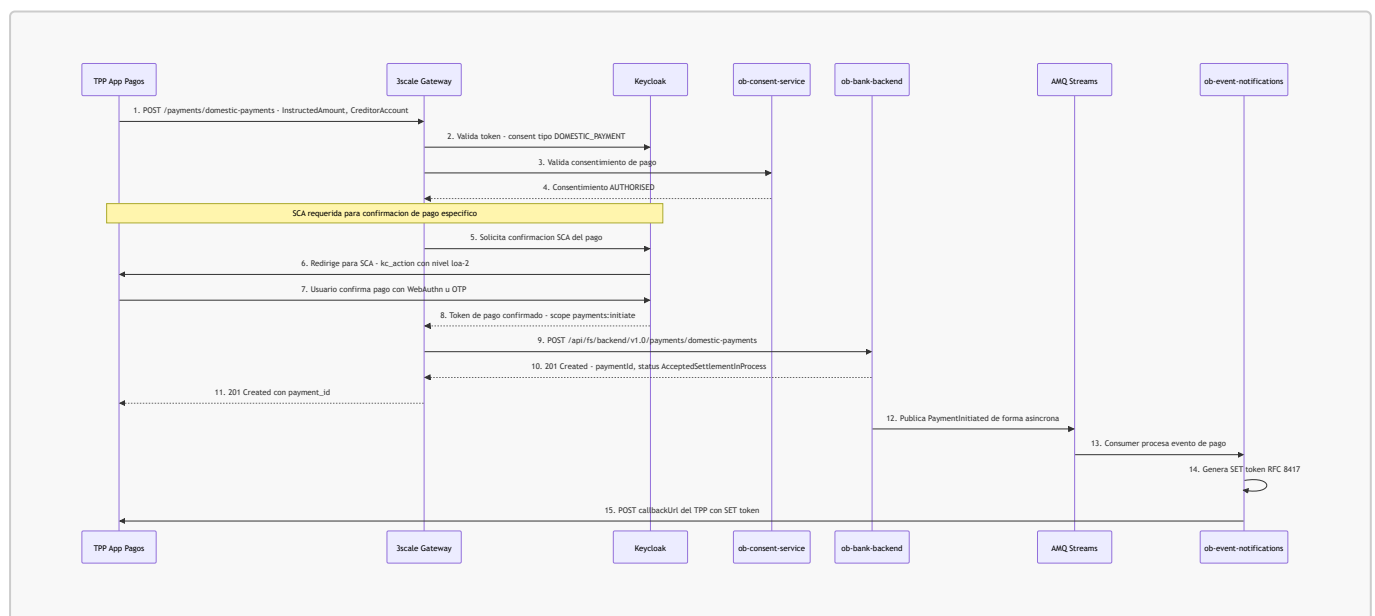
Una vez que el TPP tiene un **access_token** vinculado a un consentimiento autorizado, puede consumir las APIs del banco. Este flujo muestra los controles de seguridad que se aplican en **cada petición**: validación del

certificado mTLS del TPP, rate limiting, introspección del token en Keycloak, validación del estado del consentimiento y enriquecimiento de headers antes de llegar al backend. La auditoría de cada llamada se publica en Kafka de forma asíncrona para no impactar la latencia.



5.4 Flujo de Iniciación de Pago (Domestic Payment)

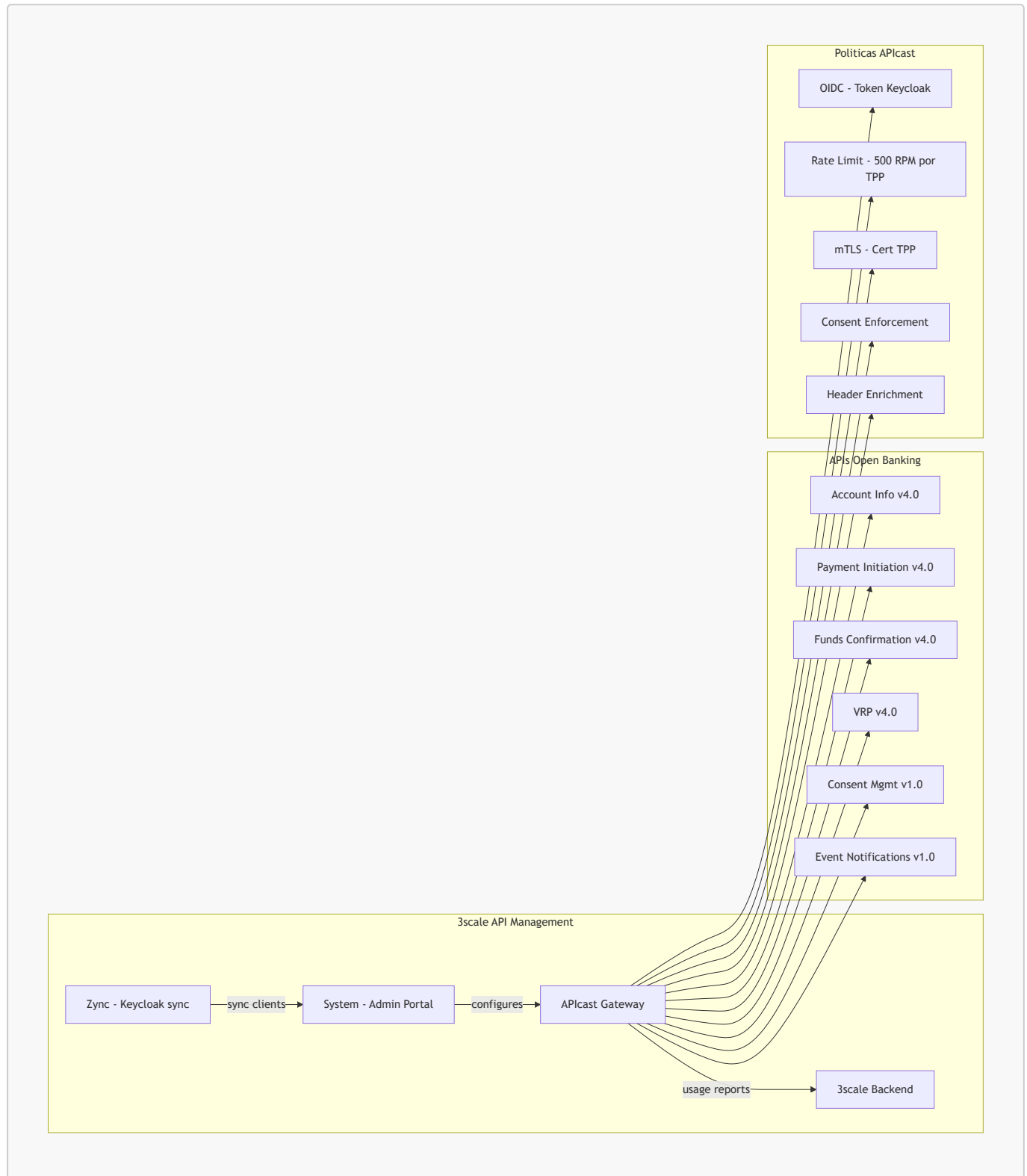
Los pagos requieren controles adicionales respecto a la consulta de cuentas. FAPI 2 exige SCA específica para cada pago — el cliente debe confirmar la operación con segundo factor incluso si ya está autenticado. Una vez procesado el pago por el backend, el estado final (**AcceptedSettlementCompleted**) se notifica al TPP de forma asíncrona mediante un **SET token** (Security Event Token, RFC 8417) enviado a su webhook registrado.



6. Diagrama de Componentes — Capa de API Management

Red Hat 3scale API Management es el corazón de la capa de gobierno de APIs. Este diagrama muestra la arquitectura interna de 3scale desplegada en OCP y su relación con las seis APIs Open Banking publicadas.

Cada API tiene asociado un conjunto de políticas que se aplican secuencialmente en el pipeline de APIcast: primero la validación mTLS del certificado del TPP, luego la validación del token OIDC contra Keycloak, después la política de consent enforcement que consulta al **ob-consent-service**, y finalmente el rate limiting que previene el abuso de la plataforma.

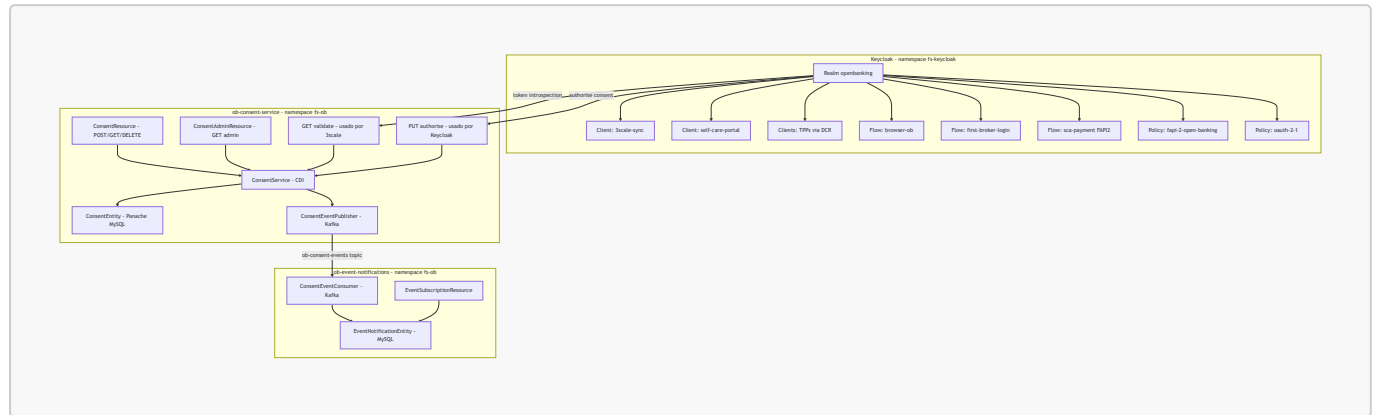


7. Diagrama de Componentes — Capa de Identidad y Consentimiento

Este diagrama detalla la relación entre Keycloak y los microservicios Quarkus de consentimiento y eventos. Keycloak es el único emisor de tokens en la plataforma — ningún otro componente emite JWTs. Los

microservicios Quarkus se comunican con Keycloak únicamente para dos propósitos: validar tokens entrantes (via `quarkus-oidc`) y notificar cambios de estado de consentimiento (via `PUT /authorise` llamado desde los authentication flows de Keycloak).

El `ob-consent-service` actúa también como componente de validación que 3scale consulta en cada petición de API. Esta arquitectura "validate-on-every-call" garantiza que si un cliente revoca un consentimiento, el bloqueo es efectivo en la siguiente petición — no al expirar el token.



8. Arquitectura de Despliegue en OCP

La arquitectura de despliegue organiza todos los componentes en **seis namespaces aislados** dentro del clúster OCP. El aislamiento por namespace tiene tres beneficios: seguridad (NetworkPolicies restringen el tráfico entre capas), operabilidad (cada equipo gestiona su namespace con su propio RBAC) y observabilidad (las métricas y logs se pueden filtrar por namespace).

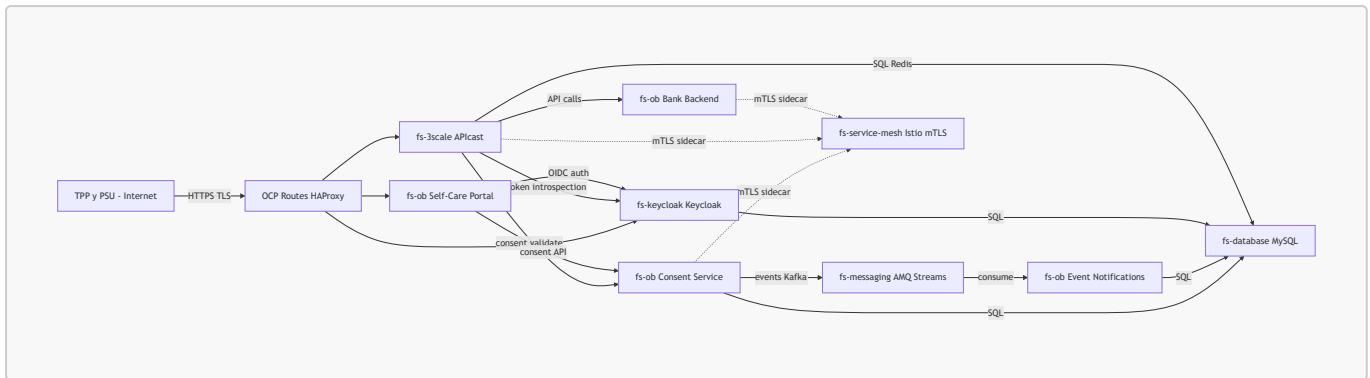
El acceso externo está centralizado en OCP Routes con TLS, que son el único vector de entrada desde internet. Los servicios internos se comunican exclusivamente a través de ClusterIP Services con mTLS habilitado por Red Hat Service Mesh (Istio). Ningún microservicio de negocio expone una Route pública directamente — todo tráfico de TPPs pasa por 3scale APIcast.

Red Hat OpenShift Container Platform 4.14

Namespace: fs-3scale	[API Management]
APIManager CR (3scale-operator)	
Deployment: apicast-production	(3 réplicas)
Deployment: apicast-staging	(1 réplica)
Deployment: system-app	(2 réplicas)
Deployment: zync	(2 réplicas)
Deployment: backend-worker	(2 réplicas)
PVC: mysql-storage	(50Gi – RWO)
PVC: redis-storage	(10Gi – RWO)
Route: api.apps.<domain>	(HTTPS – APIs externas TPP)
Route: admin.apps.<domain>	(HTTPS – Admin Portal)
Route: developer.apps.<domain>	(HTTPS – Dev Portal TPPs)
Namespace: fs-keycloak	[Identidad]
Keycloak CR (keycloak-operator)	
Deployment: keycloak	(3 réplicas – HA cluster)

└─ PVC: keycloak-data	(20Gi – RW0)
└─ Route: auth.apps.<domain>	(HTTPS – OIDC endpoints)
└─ Secret: keycloak-db-secret	
─ Namespace: fs-ob	[Open Banking Services]
└─ Deployment: ob-consent-service	(2 réplicas)
└─ Port: 8081	
└─ Env: DB_HOST, KEYCLOAK_HOST, KAFKA_BOOTSTRAP	
└─ ConfigMap: consent-config	
└─ Secret: consent-db-secret	
└─ Deployment: ob-event-notifications	(2 réplicas)
└─ Port: 8082	
└─ Secret: events-db-secret	
└─ Deployment: ob-bank-backend	(2 réplicas)
└─ Port: 8083	
└─ Deployment: self-care-portal	(2 réplicas – nginx)
└─ Port: 80	
└─ Service: ob-consent-svc	(ClusterIP)
└─ Service: ob-events-svc	(ClusterIP)
└─ Service: ob-backend-svc	(ClusterIP)
└─ Service: portal-svc	(ClusterIP)
└─ Route: portal.apps.<domain>	(HTTPS – Self-Care Portal)
─ Namespace: fs-messaging	[Mensajería]
└─ Kafka CR (amq-streams-operator)	
└─ Kafka: 3 brokers (3 réplicas)	
└─ ZooKeeper: 3 réplicas	
└─ PVC: kafka-data-* (50Gi por broker)	
└─ KafkaTopic: ob-consent-events	(3 particiones, replica 2)
└─ KafkaTopic: ob-payment-events	(3 particiones, replica 2)
└─ KafkaTopic: ob-audit-events	(6 particiones, replica 3)
─ Namespace: fs-database	[Persistencia]
└─ StatefulSet: mysql	(3 nodos – InnoDB Cluster)
└─ PVC: mysql-data-* (100Gi por nodo)	
└─ Service: mysql-svc	(ClusterIP)
─ Namespace: fs-service-mesh	[Seguridad mTLS]
└─ ServiceMeshControlPlane (istio-operator)	
└─ ServiceMeshMemberRoll: [fs-ob, fs-3scale, fs-keycloak]	
└─ PeerAuthentication: STRICT (mTLS entre pods)	
└─ AuthorizationPolicy: allow-inbound-tpp	
─ Namespace: openshift-monitoring	[Observabilidad]
└─ Prometheus (OCP Monitoring Stack)	
└─ Grafana (Community Operator + Loki datasource)	
└─ Loki + Fluentd	(Logs operacionales)
└─ Elasticsearch + Fluentd + Kibana	(EFK – Auditoria regulatoria 5 anos)
└─ Jaeger (Red Hat Service Mesh)	(Distributed Tracing)

Diagrama de comunicación entre namespaces:



9. Modelo de Datos y Bases de Datos

El modelo de datos de Open Banking es deliberadamente minimalista: los consentimientos y las notificaciones de eventos son las únicas entidades propias de la solución. El resto del estado (identidades de usuarios, clientes OAuth2, sesiones) lo gestiona Keycloak en su propia base de datos, y las métricas de API las gestiona 3scale en la suya. Esta separación evita el acoplamiento entre capas y permite escalar cada base de datos de forma independiente.

Todas las migraciones de esquema se gestionan con **Flyway**, que ejecuta automáticamente los scripts al arrancar el servicio Quarkus. Esto garantiza que el esquema de base de datos está siempre en sincronía con la versión del código desplegada, un requisito crítico en entornos con despliegues frecuentes (GitOps).

9.1 Base de datos ob_consent (Flyway V1)

```
-- Tabla principal de consentimientos
CREATE TABLE ob_consent (
  consent_id          VARCHAR(36) NOT NULL, -- UUID
  client_id           VARCHAR(255) NOT NULL, -- TPP client_id (Keycloak)
  user_id             VARCHAR(255),         -- Subject del cliente (post-
auth)
  consent_type        VARCHAR(50) NOT NULL, -- ACCOUNT_ACCESS |
DOMESTIC_PAYMENT | ...
  status              VARCHAR(30) NOT NULL, -- AWAITING_AUTHORISATION |
AUTHORISED | ...
  receipt             TEXT NOT NULL,         -- JSON payload completo del
consentimiento
  created_timestamp    DATETIME(6) NOT NULL,
  updated_timestamp    DATETIME(6) NOT NULL,
  expiration_timestamp DATETIME(6),         -- NULL = no expira
  consent_attributes   TEXT,                -- JSON metadata adicional
(permissions, etc.)
  PRIMARY KEY (consent_id),
  INDEX idx_consent_client (client_id),
  INDEX idx_consent_user   (user_id),
  INDEX idx_consent_status (status),
  INDEX idx_consent_type   (consent_type)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

9.2 Base de datos ob_events (Flyway V1)

```
-- Suscripciones webhook de TPPs para recibir eventos
CREATE TABLE ob_event_subscription (
  subscription_id VARCHAR(36) NOT NULL,
  client_id       VARCHAR(255) NOT NULL, -- TPP que se suscribe
  callback_url    VARCHAR(1024),         -- Webhook URL del TPP
  event_types     TEXT,                  -- JSON array de tipos de eventos
  version         VARCHAR(20),
  created_timestamp DATETIME(6) NOT NULL,
  PRIMARY KEY (subscription_id),
  INDEX idx_sub_client (client_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Notificaciones pendientes de entrega (SET tokens RFC 8417)
CREATE TABLE ob_event_notification (
  notification_id VARCHAR(36) NOT NULL,
  client_id       VARCHAR(255) NOT NULL,
  set_payload     TEXT NOT NULL, -- JWT Security Event Token
  status          VARCHAR(20) NOT NULL DEFAULT 'OPEN', -- OPEN | DELIVERED
  created_timestamp DATETIME(6) NOT NULL,
  PRIMARY KEY (notification_id),
  INDEX idx_notif_client (client_id),
  INDEX idx_notif_status (status)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

9.3 Bases de datos requeridas

Base de datos	Propósito	Tamaño estimado inicial
ob_consent	Consentimientos Open Banking	10 GB
ob_events	Subscripciones y notificaciones	5 GB
keycloak	Usuarios, clients, sesiones, tokens	20 GB
3scale_production	APIs, plans, metrics, apps 3scale	10 GB

10. Políticas de API (3scale)

Las políticas de 3scale son el mecanismo mediante el cual se implementan los requisitos de seguridad FAPI sin modificar el código de los microservicios backend. Funcionan como un pipeline de middleware en APIcast: cada petición entrante atraviesa las políticas en orden antes de llegar al servicio destino. Si cualquier política falla (token inválido, consentimiento revocado, rate limit excedido), la petición se rechaza con el código HTTP apropiado y se registra en el sistema de auditoría.

10.1 Política OIDC — Validación de Tokens Keycloak

Primera línea de defensa. APIcast verifica la firma del JWT usando las claves públicas de Keycloak (descargadas del endpoint JWKS), valida la expiración del token y extrae los claims relevantes (consent_id, azp, scope) que

las políticas siguientes necesitarán.

```
# APIcast Policy: OIDC
- name: oidc
  version: builtin
  configuration:
    issuer: "https://auth.apps.cluster.example.com/realms/openbanking"
    client_id: "3scale-sync"
    # Keycloak JWKS endpoint para verificación de firma JWT
    jwks_url:
      "https://auth.apps.cluster.example.com/realms/openbanking/protocol/openid-
      connect/certs"
    # Scopes requeridos por API
    required_scopes:
      - accounts:read          # Account Information API
      - payments:initiate      # Payment Initiation API
      - funds-confirmation     # Funds Confirmation API
```

10.2 Política Rate Limiting por TPP

Protege la plataforma contra el uso abusivo. El límite se aplica por `client_id` del TPP (claim `azp` del JWT), no por IP, lo que permite que un TPP con múltiples servidores comparta el mismo límite. Los límites son configurables por plan de 3scale, permitiendo diferentes cuotas para TPPs en sandbox, producción básica y producción premium.

```
# APIcast Policy: Rate Limiting
- name: rate_limit
  version: builtin
  configuration:
    limits:
      - condition: "jwt.claim.azp" # client_id del TPP desde JWT
        operations:
          - left: "200" # 200 req/min por TPP
            left_type: fixed
            op: ">="
            right: "{{http_requests_per_minute}}"
            right_type: liquid
        actions:
          - type: reject
            status: 429
            headers:
              Retry-After: "60"
              Content-Type: "application/json"
```

10.3 Política Consent Enforcement (Custom Camel K)

Esta política es la más crítica desde el punto de vista regulatorio. En cada petición de API (no solo al inicio de sesión), verifica que el consentimiento asociado al token sigue en estado `AUTHORISED` y no ha expirado. La

validación se delega al **ob-consent-service** para garantizar que una revocación por parte del cliente tiene efecto inmediato. Se implementa como una ruta **Apache Camel K** que se ejecuta como contenedor sidecar de APIcast.

```
// Ruta Camel K: ConsentEnforcementRoute.java
// Valida que el access_token tiene un consentimiento AUTHORISED asociado
@ApplicationScoped
public class ConsentEnforcementRoute extends RouteBuilder {
    @Override
    public void configure() {
        // Intercepta cada request entrante desde APIcast
        from("direct:consent-enforcement")
            .process(exchange -> {
                String consentId = exchange.getIn()
                    .getHeader("x-consent-id", String.class);
                String clientId = exchange.getIn()
                    .getHeader("x-tpi-id", String.class);

                // Llama al ob-consent-service para validar
                String url = "http://ob-consent-svc.fs-ob:8081"
                    + "/api/fs/consent/v1.0/consents/"
                    + consentId + "/validate?clientId=" + clientId;

                exchange.getIn().setHeader(Exchange.HTTP_URI, url);
            })
            .toD("http://ob-consent-svc.fs-ob:8081?bridgeEndpoint=true")
            .choice()
                .when(header("CamelHttpResponseCode").isNotEqualTo(200))
                    .setHeader(Exchange.HTTP_RESPONSE_CODE, constant(403))
                    .setBody(constant("{\"error\":\"consent_invalid\"}"))
                    .stop()
            .end();
    }
}
```

10.4 Planes de API (3scale Tiers)

Plan	Límite llamadas	Aplicación
sandbox	100 req/día	Desarrollo y testing TPPs
basic	10,000 req/día	TPPs autorizados (sandbox regulatorio)
production	100,000 req/día	TPPs con certificado regulatorio activo
premium	Sin límite	Entidades financieras asociadas

11. Consideraciones de Seguridad y FAPI

La seguridad en Open Banking no es una capa adicional — es una restricción de diseño que afecta cada componente de la arquitectura. El estándar FAPI (Financial-grade API) define requisitos de seguridad más

estrictos que los de OAuth2/OIDC convencional, específicamente diseñados para mitigar los vectores de ataque más comunes en servicios bancarios: interceptación de tokens, replay attacks, phishing de redirects y acceso no autorizado mediante tokens robados.

11.1 Perfiles FAPI implementados por Keycloak

La ventaja competitiva de usar Keycloak para Open Banking es que estos perfiles están **pre-configurados y validados** por el equipo de Keycloak contra las especificaciones de la OpenID Foundation. No es necesario implementarlos manualmente — el administrador solo necesita activar la Client Policy correspondiente en el realm.

Red Hat incluye en Keycloak 26 los siguientes perfiles listos para producción:

```
Global Client Profiles (pre-configurados):
├── fapi-1-baseline
│   ├── Executor: pkce-enforcer (S256 obligatorio)
│   ├── Executor: secure-redirect-uris (sin wildcards, HTTPS)
│   └── Executor: secure-session (duración limitada)
├── fapi-1-advanced
│   ├── Executor: pkce-enforcer
│   ├── Executor: mtls-enforcer (mTLS bound tokens)
│   ├── Executor: request-object-enforcer (JAR firmado)
│   └── Executor: par-enforcer (PAR obligatorio)
└── fapi-2-security-profile
    ├── Executor: pkce-enforcer (S256)
    ├── Executor: par-enforcer (PAR obligatorio)
    ├── Executor: dpop-enforcer (DPoP tokens)
    └── Executor: prohibit-implicit-grant (sin Implicit Flow)
```

11.2 Controles de seguridad por capa

Cada capa de la arquitectura implementa controles de seguridad independientes. Esta estrategia de **defensa en profundidad** garantiza que el compromiso de un componente no implica el compromiso de toda la plataforma. Por ejemplo, aunque un atacante lograra inyectar un token inválido, la validación de consentimiento en **ob-consent-service** lo rechazaría; aunque lograra un token válido para un consentimiento revocado, el estado en base de datos bloquearía la petición.

Capa	Control	Implementación
Perímetro	TLS 1.3 obligatorio	OCP HAProxy + Routes
API Gateway	mTLS TPP ↔ 3scale	3scale APIcast Policy
Identidad	FAPI 2 + DPoP	Keycloak Client Policy
Consentimiento	Validación en cada request	ob-consent-service validate
Inter-servicios	mTLS entre pods	Red Hat Service Mesh STRICT

Capa	Control	Implementación
Datos en tránsito	TLS en MySQL	MySQL SSL config
Datos en reposo	etcd cifrado OCP + MySQL TDE	OCP config
Secretos	OCP Secrets + etcd cifrado	OCP Secret API
Auditoría	Logs inmutables 5 años	EFK + AMQ Streams audit topic
Escaneo	CVE continuo en imágenes	Red Hat ACS (Advanced Cluster Security)

11.3 Configuración SCA en Keycloak (Authentication Flow)

```
# Browser Authentication Flow Open Banking con SCA
# Equivalente al PSD2 RTS Strong Customer Authentication
Authentication Flow: browser-ob
├── Cookie (ALTERNATIVE)           # SSO session válida → skip
├── Identity-First Login (REQUIRED)
│   ├── Username Form
│   └── SCA Sub-flow (CONDITIONAL)
│       ├── Condition: Credential Configured
│       └── Authenticators (ALTERNATIVE):
│           ├── OTP Form           # Factor 2: TOTP (Google Auth / FreeOTP)
│           └── WebAuthn Authenticator # Factor 2: FIDO2 (Passkey / Yubikey)

# Para pagos: flujo de SCA nivel 2 (LoA = 2)
Client Policy FAPI 2:
- Condition: Grant Type = authorization_code
- Condition: Scope = payments:initiate
- Action: require-acr-values = [loa-2] # Fuerza SCA
```

12. Escalabilidad y Alta Disponibilidad

Los servicios Open Banking son servicios críticos de infraestructura financiera — su indisponibilidad tiene consecuencias regulatorias directas. La SFC y las normas PSD2 exigen disponibilidad mínima del 99.5% para los endpoints de APIs bancarias. La arquitectura propuesta supera estos requisitos mediante escalado horizontal automático, múltiples réplicas con distribución geográfica dentro del clúster y mecanismos de recuperación automática.

El principio rector es: **ningún componente tiene un único punto de fallo (SPOF)**. Keycloak opera en cluster con cache Infinispan distribuida, MySQL en InnoDB Cluster de 3 nodos, Kafka con 3 brokers y factor de replicación 2, y los microservicios Quarkus con mínimo 2 réplicas distribuidas en diferentes nodos OCP.

12.1 SLA y objetivos de disponibilidad

Servicio	SLA	RPO	RTO	Réplicas mínimas
3scale APIcast (Gateway)	99.9%	15 min	30 min	3

Servicio	SLA	RPO	RTO	Réplicas mínimas
Keycloak	99.9%	15 min	30 min	3
ob-consent-service	99.5%	1 hora	30 min	2
ob-event-notifications	99.5%	1 hora	1 hora	2
ob-bank-backend	99.5%	N/A (stateless)	5 min	2
MySQL InnoDB Cluster	99.9%	5 min	15 min	3
AMQ Streams (Kafka)	99.9%	1 min	5 min	3 brokers

12.2 Horizontal Pod Autoscaler para servicios Quarkus

```
# HPA – ob-consent-service
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: ob-consent-service-hpa
  namespace: fs-ob
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: ob-consent-service
  minReplicas: 2
  maxReplicas: 8
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 80
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 30
      policies:
        - type: Pods
          value: 2
          periodSeconds: 60
    scaleDown:
      stabilizationWindowSeconds: 300
```

12.3 Alta disponibilidad de Keycloak (Infinispan distributed cache)

```
# Keycloak CR – HA con cache distribuida
apiVersion: k8s.keycloak.org/v2alpha1
kind: Keycloak
metadata:
  name: keycloak-openbanking
  namespace: fs-keycloak
spec:
  instances: 3
  db:
    vendor: mysql
    host: mysql-svc.fs-database
    database: keycloak
    usernameSecret:
      name: keycloak-db-secret
      key: username
    passwordSecret:
      name: keycloak-db-secret
      key: password
  http:
    tlsSecret: keycloak-tls-secret
  hostname:
    hostname: auth.apps.cluster.example.com
  cache:
    # Infinispan distribuido entre réplicas (JGroups KUBE_PING)
    configMapFile:
      name: keycloak-cache-config
      key: cache-ispn.xml
  resources:
    requests:
      memory: "512Mi"
      cpu: "500m"
    limits:
      memory: "1Gi"
      cpu: "2"
```

12.4 Estrategia de despliegue GitOps (ArgoCD)

```
# Application ArgoCD – Open Banking Services
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: open-banking-rh
  namespace: openshift-gitops
spec:
  project: financial-services
  source:
    repoURL: https://github.com/garzontaleroj/financial-services-accelerator-local
    targetRevision: main
```

```

    path: open-banking-rh/deploy/overlays/production
  destination:
    server: https://kubernetes.default.svc
    namespace: fs-ob
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
      - ApplyOutOfSyncOnly=true

```

13. Manifiestos OCP de Referencia

Esta sección proporciona los manifiestos Kubernetes/OCP de referencia para los componentes principales de la solución. Estos manifiestos están diseñados para entornos de producción: incluyen configuración de seguridad (`securityContext` con principio de mínimo privilegio), health checks correctamente configurados para Quarkus, inyección de secretos desde OCP Secrets (no variables hardcodeadas) e integración con el Service Mesh de Istio.

En una implementación real, estos manifiestos se gestionarían con **Kustomize** (overlays por entorno: dev/staging/prod) y se desplegarían mediante **ArgoCD** en un flujo GitOps donde cualquier cambio en el repositorio Git se refleja automáticamente en el clúster.

13.1 Deployment — ob-consent-service

Este Deployment define las especificaciones del pod para `ob-consent-service`. Puntos clave: la anotación `sidecar.istio.io/inject: true` activa el sidecar de Istio para mTLS automático; el `securityContext` elimina todos los privilegios del proceso; los health checks usan el endpoint `/q/health` nativo de Quarkus SmallRye Health; las credenciales de base de datos se inyectan desde un OCP Secret en lugar de estar en el manifiesto.

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: ob-consent-service
  namespace: fs-ob
  labels:
    app: ob-consent-service
    version: "1.0.0"
    app.kubernetes.io/part-of: open-banking-rh
spec:
  replicas: 2
  selector:
    matchLabels:
      app: ob-consent-service
  template:
    metadata:
      labels:

```

```
  app: ob-consent-service
  version: "1.0.0"
  annotations:
    sidecar.istio.io/inject: "true"      # Service Mesh mTLS
spec:
  serviceAccountName: ob-consent-sa
  containers:
  - name: ob-consent-service
    image: quay.io/openbanking-rh/ob-consent-service:1.0.0
    imagePullPolicy: Always
    ports:
    - containerPort: 8081
      name: http
    env:
    - name: QUARKUS_DATASOURCE_JDBC_URL
      value: "jdbc:mysql://mysql-svc.fs-database:3306/ob_consent?useSSL=true"
    - name: QUARKUS_DATASOURCE_USERNAME
      valueFrom:
        secretKeyRef:
          name: consent-db-secret
          key: username
    - name: QUARKUS_DATASOURCE_PASSWORD
      valueFrom:
        secretKeyRef:
          name: consent-db-secret
          key: password
    - name: QUARKUS_OIDC_AUTH_SERVER_URL
      value: "https://auth.apps.cluster.example.com/realms/openbanking"
    - name: KAFKA_BOOTSTRAP_SERVERS
      value: "ob-kafka-kafka-bootstrap.fs-messaging:9092"
  resources:
    requests:
      memory: "64Mi"
      cpu: "100m"
    limits:
      memory: "256Mi"
      cpu: "500m"
  readinessProbe:
    httpGet:
      path: /q/health/ready
      port: 8081
    initialDelaySeconds: 15
    periodSeconds: 10
  livenessProbe:
    httpGet:
      path: /q/health/live
      port: 8081
    initialDelaySeconds: 30
    periodSeconds: 20
  securityContext:
    runAsNonRoot: true
    readOnlyRootFilesystem: true
    allowPrivilegeEscalation: false
```

```
capabilities:
  drop: ["ALL"]
```

13.2 Service + Route — ob-consent-service

El Service expone el pod internamente en el clúster (ClusterIP). La Route tiene una restricción de IP whitelist que limita el acceso a la red interna del clúster (**10.0.0.0/8**), garantizando que ningún TPP externo puede acceder directamente al **ob-consent-service** — deben pasar obligatoriamente por 3scale APIcast, donde se aplican todas las políticas de seguridad.

```
---
apiVersion: v1
kind: Service
metadata:
  name: ob-consent-svc
  namespace: fs-ob
spec:
  selector:
    app: ob-consent-service
  ports:
    - port: 8081
      targetPort: 8081
      name: http
---
# Route expuesta solo internamente (sin acceso desde internet)
# El acceso externo es SOLO a través de 3scale APIcast
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: ob-consent-internal
  namespace: fs-ob
  annotations:
    haproxy.router.openshift.io/ip_whitelist: "10.0.0.0/8" # Solo red interna
spec:
  host: consent-internal.apps.cluster.example.com
  to:
    kind: Service
    name: ob-consent-svc
  tls:
    termination: edge
    insecureEdgeTerminationPolicy: Redirect
```

13.3 NetworkPolicy — aislamiento entre namespaces

Las NetworkPolicies implementan el modelo de **zero-trust networking** entre namespaces de OCP. Por defecto, todos los pods del namespace **fs-ob** rechazan tráfico entrante de cualquier origen. Solo se permite tráfico desde los namespaces **fs-3scale** (APIcast), **fs-keycloak** (auth flows) y **fs-service-mesh** (Istio sidecars). Este control es la salvaguarda final que garantiza que incluso si un atacante comprometiera un pod en otro namespace, no podría acceder a los servicios Open Banking directamente.

```
# Solo 3scale y Keycloak pueden llamar a los servicios Quarkus
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-apicast-and-keycloak
  namespace: fs-ob
spec:
  podSelector:
    matchLabels:
      app.kubernetes.io/part-of: open-banking-rh
  policyTypes:
  - Ingress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          name: fs-3scale      # 3scale APIcast
    - namespaceSelector:
        matchLabels:
          name: fs-keycloak    # Keycloak auth flows
    - namespaceSelector:
        matchLabels:
          name: fs-service-mesh # Istio sidecar
  ports:
  - port: 8081    # ob-consent-service
  - port: 8082    # ob-event-notifications
  - port: 8083    # ob-bank-backend
  - port: 80      # self-care-portal
```

13.4 KafkaTopic — ob-consent-events

Este manifiesto de AMQ Streams (Strimzi Operator) define el topic Kafka para eventos de consentimiento. La configuración de `min.insync.replicas: 2` garantiza que un mensaje solo se confirma como escrito cuando al menos 2 de los 3 brokers lo han persistido, lo que protege contra la pérdida de eventos en caso de fallo de un broker. La retención de 7 días permite reintentos de procesamiento si `ob-event-notifications` tiene una interrupción temporal.

```
apiVersion: kafka.strimzi.io/v1beta2
kind: KafkaTopic
metadata:
  name: ob-consent-events
  namespace: fs-messaging
  labels:
    strimzi.io/cluster: ob-kafka
spec:
  partitions: 3
  replicas: 2
  config:
    retention.ms: 604800000    # 7 días
    segment.bytes: 1073741824  # 1 GB por segmento
```



```
cleanup.policy: delete  
min.insync.replicas: "2" # Mínimo 2 réplicas in-sync
```

Referencias y recursos adicionales

Red Hat — Documentación oficial Open Banking / FAPI

Recurso	URL
Keycloak FAPI Support	https://www.keycloak.org/docs/latest/securing_apps/index.html#_fapi-support
3scale API Management	https://access.redhat.com/documentation/en-us/red_hat_3scale_api_management
Red Hat Service Mesh	https://docs.openshift.com/container-platform/latest/service_mesh/v2x/ossm-about.html
Quarkus OIDC	https://quarkus.io/guides/security-oidc-code-flow-authentication
AMQ Streams (Kafka)	https://access.redhat.com/documentation/en-us/red_hat_amq_streams
Loki on OCP	https://docs.openshift.com/container-platform/latest/observability/logging/log_storage/installing-log-storage.html
Jaeger Distributed Tracing	https://docs.openshift.com/container-platform/latest/observability/distr_tracing/distr_tracing_jaeger/distr-tracing-jaeger-installing.html
Red Hat Financial Services	https://www.redhat.com/en/solutions/financial-services

Estándares Open Banking implementados

Estándar	Organismo	Implementación
Open Banking UK v4.0	OBIE	ob-bank-backend (API contracts)
FAPI 1.0 Advanced	OpenID Foundation	Keycloak Client Policies
FAPI 2.0 Security Profile	OpenID Foundation	Keycloak Client Policies
RFC 7591/7592 (DCR)	IETF	Keycloak nativo
RFC 9126 (PAR)	IETF	Keycloak nativo
RFC 9449 (DPoP)	IETF	Keycloak (feature dpop)
RFC 8417 (SET)	IETF	ob-event-notifications
PSD2 RTS (SCA)	EBA	Keycloak Auth Flows WebAuthn

Estándar	Organismo	Implementación
ISO 20022	ISO	Payload de pagos (ob-bank-backend)

Documento generado el 2026-06-25. Próxima revisión: 2026-07-27.
Contacto técnico: Juan Pablo Garzon — Ticxar / Red Hat Financial Services LAM.